

// Jivops

Guía completa — 30 lecciones

Fundamentos · Escaneo de imágenes · Más allá de imágenes ·
Integración CI/CD · Operación a escala · Seguridad avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Trivy y qué tipos de escaneo soporta
- Lección 02 — Instalación de Trivy (CLI, servidor)
- Lección 03 — Trivy en modo standalone vs client/server
- Lección 04 — Bases de datos de vulnerabilidades (vuln-db)
- Lección 05 — Formatos de salida (JSON, SARIF, table)

Módulo 2: Escaneo de imágenes

- Lección 06 — Escaneo de imágenes Docker/OCI
- Lección 07 — Severidad de vulnerabilidades — filtrado
- Lección 08 — Escaneo de capas (layers) de una imagen
- Lección 09 — Ignorar vulnerabilidades específicas (.trivyignore)
- Lección 10 — Escaneo de imágenes distroless/scratch

Módulo 3: Escaneo más allá de imágenes

- Lección 11 — Escaneo de sistemas de archivos (filesystem)
- Lección 12 — Escaneo de repositorios Git
- Lección 13 — Escaneo de IaC (Terraform, Kubernetes manifests)
- Lección 14 — Escaneo de dependencias de aplicación (SCA)
- Lección 15 — Escaneo de licencias de software

Módulo 4: Integración CI/CD

- Lección 16 — Integración con GitHub Actions
- Lección 17 — Integración con GitLab CI
- Lección 18 — Integración con Jenkins
- Lección 19 — Fail build por severidad — políticas de umbral
- Lección 20 — Trivy como admission controller en Kubernetes

Módulo 5: Operación a escala

- Lección 21 — Trivy Operator en Kubernetes
- Lección 22 — Escaneo continuo de imágenes en el cluster
- Lección 23 — Integración con Artifactory/registries privados
- Lección 24 — Caché de vulnerabilidades y rendimiento
- Lección 25 — Actualización de la base de datos en entornos air-gapped

Módulo 6: Seguridad avanzada y madurez

Lección 26 — SBOM (Software Bill of Materials) con Trivy

Lección 27 — Firma y verificación de SBOM

Lección 28 — Integración con políticas OPA/Gatekeeper

Lección 29 — Reportes y dashboards de vulnerabilidades

Lección 30 — Proyecto final: pipeline de seguridad completo con Trivy

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Trivy y qué tipos de escaneo soporta

Trivy escanea imágenes, filesystem, repos Git, IaC y dependencias. Un solo escáner para todo reduce las herramientas distintas que el equipo debe mantener.

Lección 02 — Instalación de Trivy (CLI, servidor)

Tener Trivy instalado localmente permite detectar vulnerabilidades de forma temprana, antes incluso de hacer commit o abrir un pull request.

```
brew install trivy
```

Lección 03 — Trivy en modo standalone vs client/server

El modo servidor cachea la vuln-db centralizadamente, evitando que cada ejecución de CI/CD la descargue por su cuenta.

```
trivy server --listen 0.0.0.0:4954
```

Lección 04 — Bases de datos de vulnerabilidades (vuln-db)

Mantener la vuln-db actualizada es crítico: aparecen vulnerabilidades nuevas constantemente que una base desactualizada no podría detectar.

Lección 05 — Formatos de salida (JSON, SARIF, table)

SARIF es un formato estándar que plataformas como GitHub interpretan directamente en su interfaz de seguridad, sin parsing adicional.

```
trivy image --format sarif
```

Módulo 2: Escaneo de imágenes

Lección 06 — Escaneo de imágenes Docker/OCI

Trivy analiza las capas de una imagen buscando paquetes vulnerables. Escanear antes del despliegue evita exponer el problema en un entorno real.

```
trivy image nginx:latest
```

Lección 07 — Severidad de vulnerabilidades — filtrado

Filtrar por CRITICAL y HIGH enfoca la atención del equipo en lo urgente, evitando que alertas de bajo impacto saturen y oculten lo importante.

```
trivy image --severity CRITICAL,HIGH
```

Lección 08 — Escaneo de capas (layers) de una imagen

Identificar en qué capa se introdujo una vulnerabilidad ayuda a rastrear el origen hasta una instrucción específica del Dockerfile.

Lección 09 — Ignorar vulnerabilidades específicas (.trivyignore)

Documentar la razón al ignorar un CVE es clave: sin justificación, alguien revisando después no sabría si fue una decisión consciente o un descuido.

Lección 10 — Escaneo de imágenes distroless/scratch

Aunque reducen la superficie de ataque del SO, las dependencias de la aplicación empaquetadas dentro pueden seguir teniendo vulnerabilidades.

Módulo 3: Escaneo más allá de imágenes

Lección 11 — Escaneo de sistemas de archivos (filesystem)

'trivy fs' escanea un directorio local directamente, detectando dependencias vulnerables antes incluso de construir la imagen Docker completa.

```
trivy fs ./mi-proyecto
```

Lección 12 — Escaneo de repositorios Git

'trivy repo' clona y escanea por URL directamente, ahorrando el paso manual de clonar cada repositorio antes de poder escanearlo.

```
trivy repo https://github.com/miorg/repo
```

Lección 13 — Escaneo de IaC (Terraform, Kubernetes manifests)

Detecta configuraciones inseguras como un bucket S3 público antes de aplicar el Terraform, evitando exponer datos reales tras el despliegue.

```
trivy config ./terraform/
```

Lección 14 — Escaneo de dependencias de aplicación (SCA)

La mayoría de vulnerabilidades en apps modernas vienen de dependencias de terceros, que amplían enormemente la superficie más allá del código propio.

Lección 15 — Escaneo de licencias de software

Detecta licencias incompatibles (como GPL en software propietario), un riesgo legal que podría obligar a liberar código propio.

```
trivy image --scanners license
```

Módulo 4: Integración CI/CD

Lección 16 — Integración con GitHub Actions

La Action oficial simplifica ejecutar Trivy sin instalar manualmente el binario en cada ejecución del workflow.

```
uses: aquasecurity/trivy-action@master
```

Lección 17 — Integración con GitLab CI

La plantilla integrada muestra resultados directamente en el dashboard de seguridad de GitLab, sin parsing manual de logs.

Lección 18 — Integración con Jenkins

Usar `--exit-code` para que Trivy retorne un código distinto de cero es clave para que Jenkins detecte el fallo y bloquee el pipeline automáticamente.

```
sh 'trivy image --exit-code 1 miapp:latest'
```

Lección 19 — Fail build por severidad — políticas de umbral

Definir un umbral de severidad para fallar (en vez de cualquier vulnerabilidad) es más práctico, evitando fricción constante por hallazgos menores.

```
trivy image --severity CRITICAL --exit-code 1
```

Lección 20 — Trivy como admission controller en Kubernetes

Actúa como última línea de defensa dentro del cluster, protegiendo incluso si alguien despliega sin pasar por el pipeline de CI/CD habitual.

Módulo 5: Operación a escala

Lección 21 — Trivy Operator en Kubernetes

Escanea continuamente workloads ya desplegados, generando VulnerabilityReports como recursos de Kubernetes, útil aunque la imagen no haya cambiado.

Lección 22 — Escaneo continuo de imágenes en el cluster

Una imagen 'segura' hace 3 meses podría dejar de serlo hoy si se descubrió una vulnerabilidad nueva en algún paquete, sin que la imagen misma cambiara.

Lección 23 — Integración con Artifactory/registries privados

Usar una credencial de solo lectura para que Trivy acceda al registry es más seguro que una cuenta con permisos de escritura completos.

```
TRIVY_USERNAME / TRIVY_PASSWORD
```

Lección 24 — Caché de vulnerabilidades y rendimiento

Compartir la caché entre varios runners de CI/CD evita que cada uno descargue por separado la misma base de datos, ahorrando tiempo agregado.

```
trivy image --cache-dir /tmp/trivy-cache
```

Lección 25 — Actualización de la base de datos en entornos air-gapped

Una vuln-db desactualizada por mucho tiempo en un entorno air-gapped da una falsa sensación de seguridad: los escaneos corren 'bien' sin detectar lo nuevo.

```
trivy image --offline-scan
```


Módulo 6: Seguridad avanzada y madurez

Lección 26 — SBOM (Software Bill of Materials) con Trivy

Un SBOM completo permite verificar rápidamente si una librería vulnerable recién anunciada está presente en la aplicación, sin investigar desde cero.

```
trivy image --format cyclonedx
```

Lección 27 — Firma y verificación de SBOM

Verificar la firma de un SBOM antes de confiar en su contenido evita que alguien lo haya manipulado para ocultar un componente vulnerable.

Lección 28 — Integración con políticas OPA/Gatekeeper

OPA permite lógica de política personalizada más allá de un umbral fijo, como reglas distintas según namespace o equipo.

Lección 29 — Reportes y dashboards de vulnerabilidades

Un dashboard centralizado da visibilidad comparativa entre proyectos, ayudando a priorizar dónde invertir recursos de seguridad.

Lección 30 — Proyecto final: pipeline de seguridad completo con Trivy

Se integra todo lo anterior: escaneo de imágenes/IaC/dependencias en CI/CD, umbral de severidad, admission controller, Trivy Operator, SBOM y dashboards — el nivel esperado de un pipeline de seguridad listo para producción real.